

THE STENCIL METHOD

Ahmed Tarek

INTRODUCTION

PROBLEM

The 2D heat equation needs to be solved on a large grid using finite differences with a 5-point stencil. Serial computation is too slow we need parallel processing to handle realistic grid sizes efficiently.

GOAL

Implement a parallel version using MPI (distributed memory across nodes)+ OpenMP (shared memory within nodes) that solves the heat equation faster by distributing the grid across multiple processors, while maintaining correctness and measuring parallel efficiency.

THE STEPS

1. Grid Decomposition
2. Ghost Layers & Border Exchange
3. OpenMP Loop Parallelization
4. MPI Communication Pattern
5. Performance Instrumentation (results)
6. Scalability Testing (results)

GRID DECOMPOSITION

What is it?

Divide the 2D computational grid into rectangular blocks. Each MPI task gets one block and is responsible for computing updates on that block.

Why do this?

- Distributes work across multiple processors
- Each processor works on a smaller grid → faster computation
- Allows different processors to compute in parallel

GRID DECOMPOSITION

How does it work?

- Original grid size: $N \times N$ (e.g., $10,000 \times 10,000$)
- Number of MPI tasks arranged in a 2D grid: $P_x \times P_y$ (e.g., $4 \times 4 = 16$ tasks)
- Each task gets a block of size: $(N/P_x) \times (N/P_y)$
- Example: $10,000 \times 10,000$ grid split into 4×4 tasks $\rightarrow$ each task handles $2,500 \times 2,500$ grid points

Key idea:

Task (ij) computes only its own rectangular region. It does NOT need to know about the entire global grid.

```
GRIDSIZE=20000
ITERATIONS=1000
# Node counts 1*1 2*2 4*4
NODE_COUNTS="1 2 4"
```


GHOST LAYERS & BORDER EXCHANGE

What is it?

Each MPI task keeps a copy of the boundary values from its neighboring tasks. These are called "ghost layers" or "halo regions."

Why do we need this?

When a grid point is at the edge of a task's domain, its neighbors belong to neighboring tasks. Without ghost layers, we couldn't compute the stencil.

GHOST LAYERS & BORDER EXCHANGE

How does it work?

1. Each task stores its own grid block PLUS a 1-point thick border on all 4 sides (north, south, east, west)
2. Before each update, tasks exchange their boundary values with neighbors
3. After exchange, each task has up-to-date ghost values and can safely update its interior

OPENMP STEP

what is OpenMP?

OpenMP (Open Multi-Processing) is a shared-memory parallel programming model that uses compiler directives to parallelize loops within each MPI task.

For stencil codes, it splits the local grid update loop among multiple threads running on the same node

Why is this important?

- Reduces MPI overhead: Fewer MPI tasks per node = less communication overhead
- Better memory bandwidth: Threads share the same physical memory (cache-friendly)
- Hybrid scaling: Combines distributed-memory (MPI) with shared-memory (OpenMP) parallelism

OPENMP STEP

```
// East/West: copy to buffers (non-contiguous)
#pragma omp parallel for
for (int j = 1; j <= sizey; j++) {
    buffers[SEND][WEST][j-1] = current_plane[j * full_size_x + 1];
    buffers[SEND][EAST][j-1] = current_plane[j * full_size_x + size_x];
}
```

Pack East/West Boundaries

Parallel packing before MPI

Static schedule (default) for uniform workload - each iteration does 2 memory reads/writes.

Parallelizes buffer preparation before non-blocking MPI_Isend.

OPENMP STEP

```
/* Copy East/West halos back */  
#pragma omp parallel for  
for (int j = 1; j <= sizey; j++) {  
    current_plane[j * full_size_x] = buffers[RECV][WEST][j-1];  
    current_plane[j * full_size_x + size_x + 1] = buffers[RECV][EAST][j-1];  
}
```

Unpack East/West Halos

After MPI_Waitall

Threads copy ghost cell data from receive buffers back into the plane's halo layers.

No race conditions - each j writes to different locations.

OPENMP STEP

```
/* Non-periodic boundaries: set to 0 */
if (!periodic) {
    if (neighbours[NORTH] == MPI_PROC_NULL) {
        #pragma omp parallel for
        for (int i = 0; i < full_size; i++)
            current_plane[i] = 0.0;
    }
    if (neighbours[SOUTH] == MPI_PROC_NULL) {
        #pragma omp parallel for
        for (int i = 0; i < full_size; i++)
            current_plane[(size + 1) * full_size + i] = 0.0;
    }
    if (neighbours[WEST] == MPI_PROC_NULL) {
        #pragma omp parallel for
        for (int j = 0; j < full_size; j++)
            current_plane[j * full_size] = 0.0;
    }
    if (neighbours[EAST] == MPI_PROC_NULL) {
        #pragma omp parallel for
        for (int j = 0; j < full_size; j++)
            current_plane[j * full_size + size + 1] = 0.0;
    }
}
```

Boundary Conditions

Small loop parallelization

OpenMP overhead may exceed benefit.

Consider serial or `#pragma omp simd` instead for tiny boundary loops.

MPI STEP

what is MPI?

MPI enables multiple independent processes (not threads) to work together on a problem by explicitly exchanging messages. Before each update, tasks exchange their boundary values with neighbors.

MPI: Separate processes, separate memory spaces, communicate via messages

MPI STEP

```
double* current_plane = planes[current].data;
const int sizex = planes[current].size[_x_];
const int sizey = planes[current].size[_y_];
const int full_sizex = sizex + 2;
const int full_sizey = sizey + 2;

// North/South: point directly to data (contiguous)
buffers[SEND][NORTH] = current_plane + full_sizex;
buffers[SEND][SOUTH] = current_plane + sizey * full_sizex;
buffers[RECV][NORTH] = current_plane;
buffers[RECV][SOUTH] = current_plane + (sizey + 1) * full_sizex;
```

Blocking Version

This section prepares the data to be sent—it sets up pointers to boundary rows/columns

MPI STEP

```
/* Non-blocking communication */
MPI_Request reqs[8];
int req_idx = 0;

if (neighbours[NORTH] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][NORTH], size, MPI_DOUBLE, neighbours[NORTH], 0, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][NORTH], size, MPI_DOUBLE, neighbours[NORTH], 1, myCOMM_WORLD, &reqs[req_idx++]);
}

if (neighbours[SOUTH] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][SOUTH], size, MPI_DOUBLE, neighbours[SOUTH], 1, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][SOUTH], size, MPI_DOUBLE, neighbours[SOUTH], 0, myCOMM_WORLD, &reqs[req_idx++]);
}

if (neighbours[WEST] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][WEST], size, MPI_DOUBLE, neighbours[WEST], 2, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][WEST], size, MPI_DOUBLE, neighbours[WEST], 3, myCOMM_WORLD, &reqs[req_idx++]);
}

if (neighbours[EAST] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][EAST], size, MPI_DOUBLE, neighbours[EAST], 3, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][EAST], size, MPI_DOUBLE, neighbours[EAST], 2, myCOMM_WORLD, &reqs[req_idx++]);
}

MPI_Waitall(req_idx, reqs, MPI_STATUSES_IGNORE);

total_comm_time += MPI_Wtime() - section_start;
```

Non-Blocking Version

send and receive operations for all 4 neighbors simultaneously they all return immediately, letting you continue to computation while MPI handles the transfers in background.

MPI STEP

```
/* Non-blocking communication */
MPI_Request reqs[8];
int req_idx = 0;

if (neighbours[NORTH] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][NORTH], size, MPI_DOUBLE, neighbours[NORTH], 0, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][NORTH], size, MPI_DOUBLE, neighbours[NORTH], 1, myCOMM_WORLD, &reqs[req_idx++]);
}

if (neighbours[SOUTH] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][SOUTH], size, MPI_DOUBLE, neighbours[SOUTH], 1, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][SOUTH], size, MPI_DOUBLE, neighbours[SOUTH], 0, myCOMM_WORLD, &reqs[req_idx++]);
}

if (neighbours[WEST] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][WEST], size, MPI_DOUBLE, neighbours[WEST], 2, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][WEST], size, MPI_DOUBLE, neighbours[WEST], 3, myCOMM_WORLD, &reqs[req_idx++]);
}

if (neighbours[EAST] != MPI_PROC_NULL) {
    MPI_Isend(bufs[SEND][EAST], size, MPI_DOUBLE, neighbours[EAST], 3, myCOMM_WORLD, &reqs[req_idx++]);
    MPI_Irecv(bufs[RECV][EAST], size, MPI_DOUBLE, neighbours[EAST], 2, myCOMM_WORLD, &reqs[req_idx++]);
}

MPI_Waitall(req_idx, reqs, MPI_STATUSES_IGNORE);

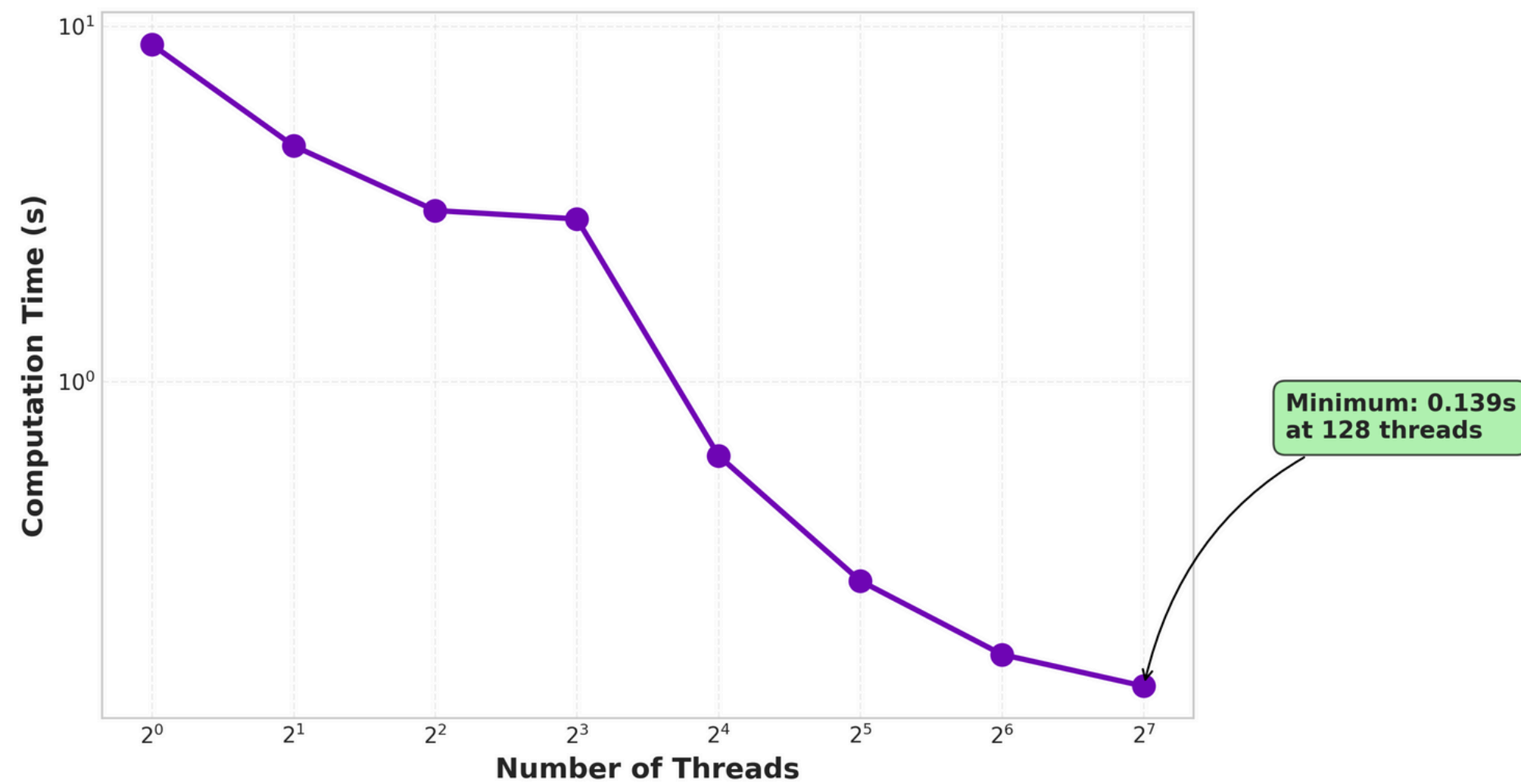
total_comm_time += MPI_Wtime() - section_start;
```

Non-Blocking Version

send and receive operations for all 4 neighbors simultaneously they all return immediately, letting you continue to computation while MPI handles the transfers in background.

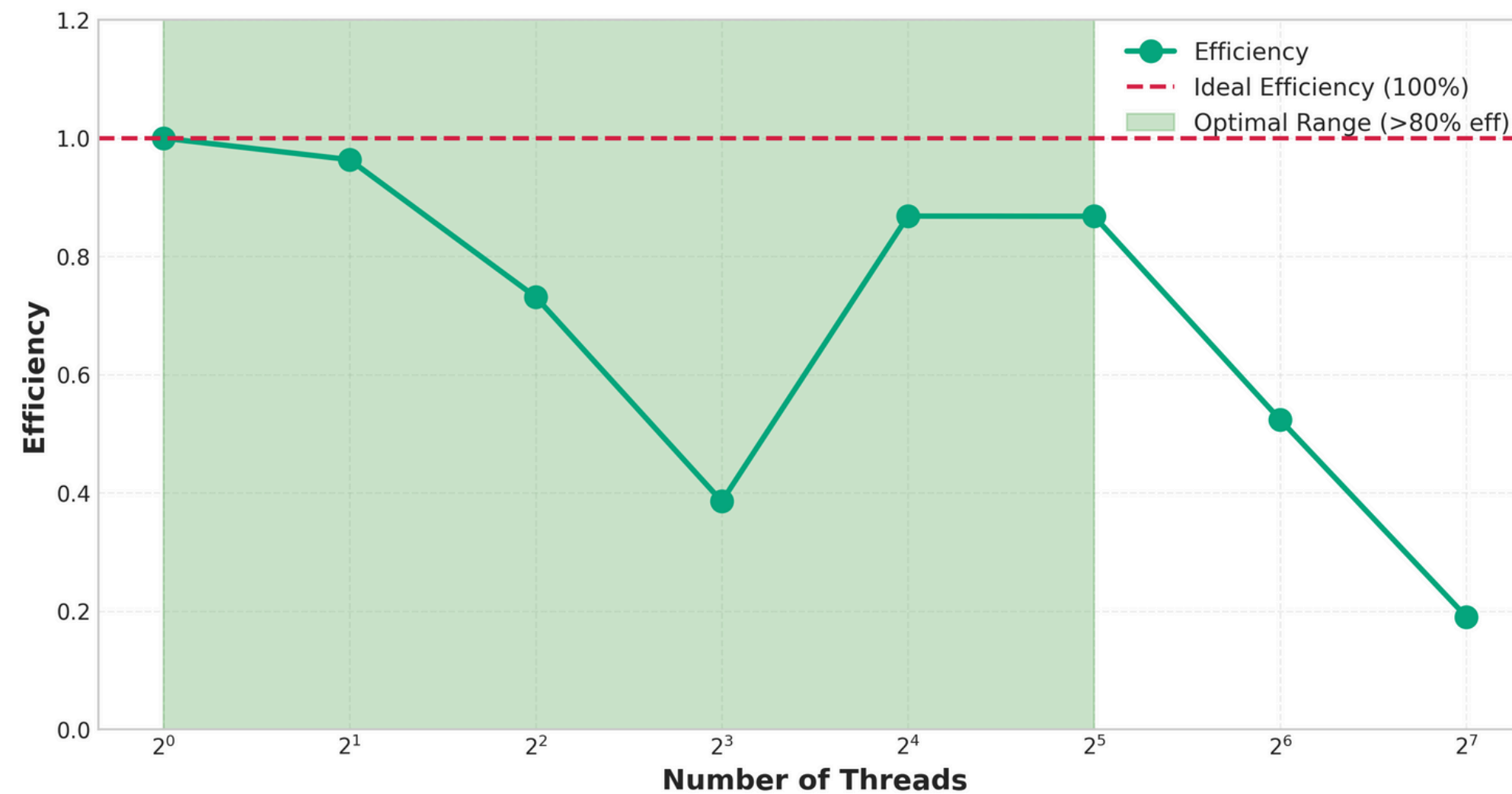
OPEN MP RESULTS

OpenMP Computation Time vs Number of Threads
1 Node, 1 MPI Task, Variable Threads



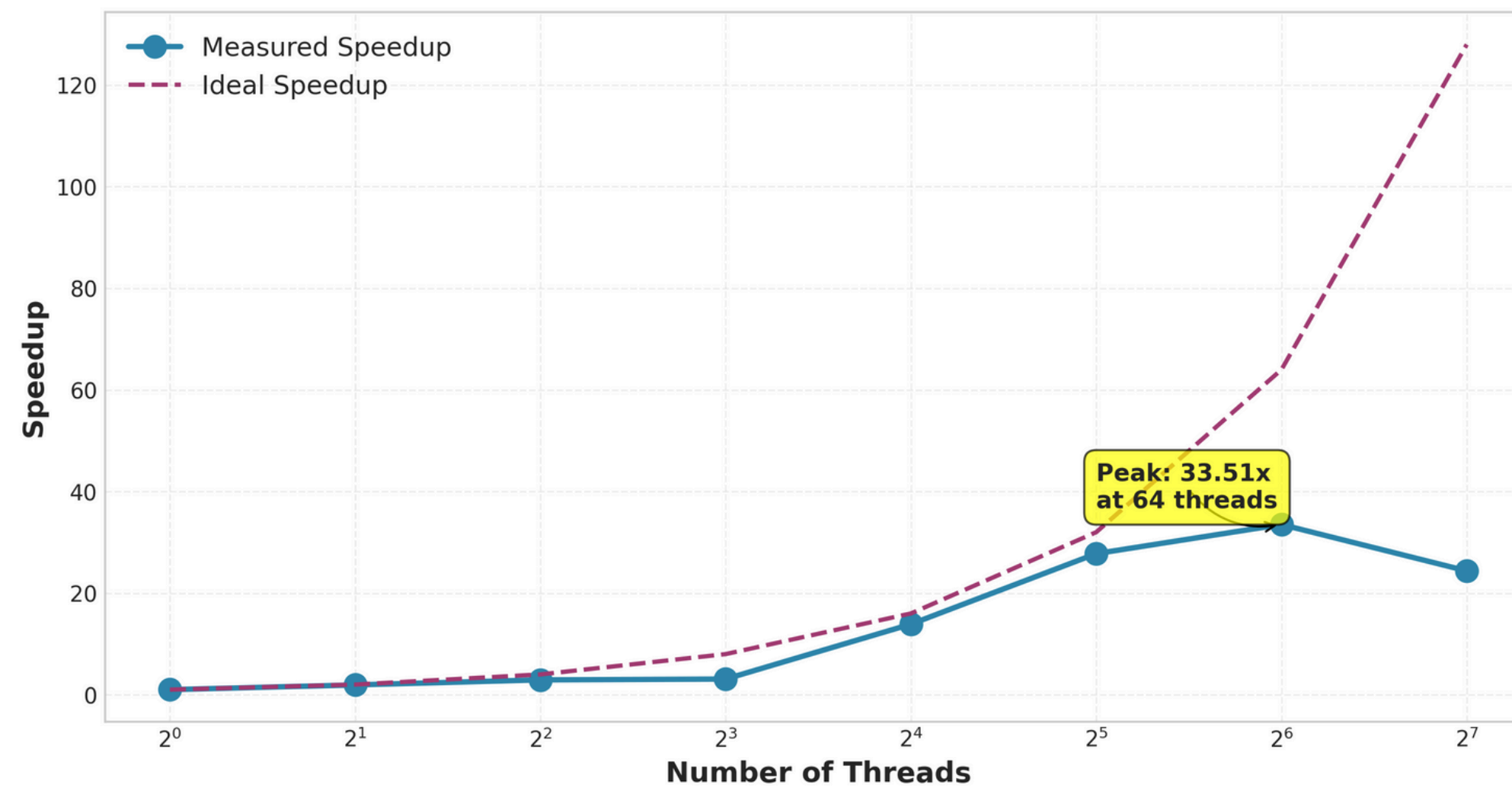
OPEN MP RESULTS

OpenMP Efficiency vs Number of Threads
1 Node, 1 MPI Task, Variable Threads



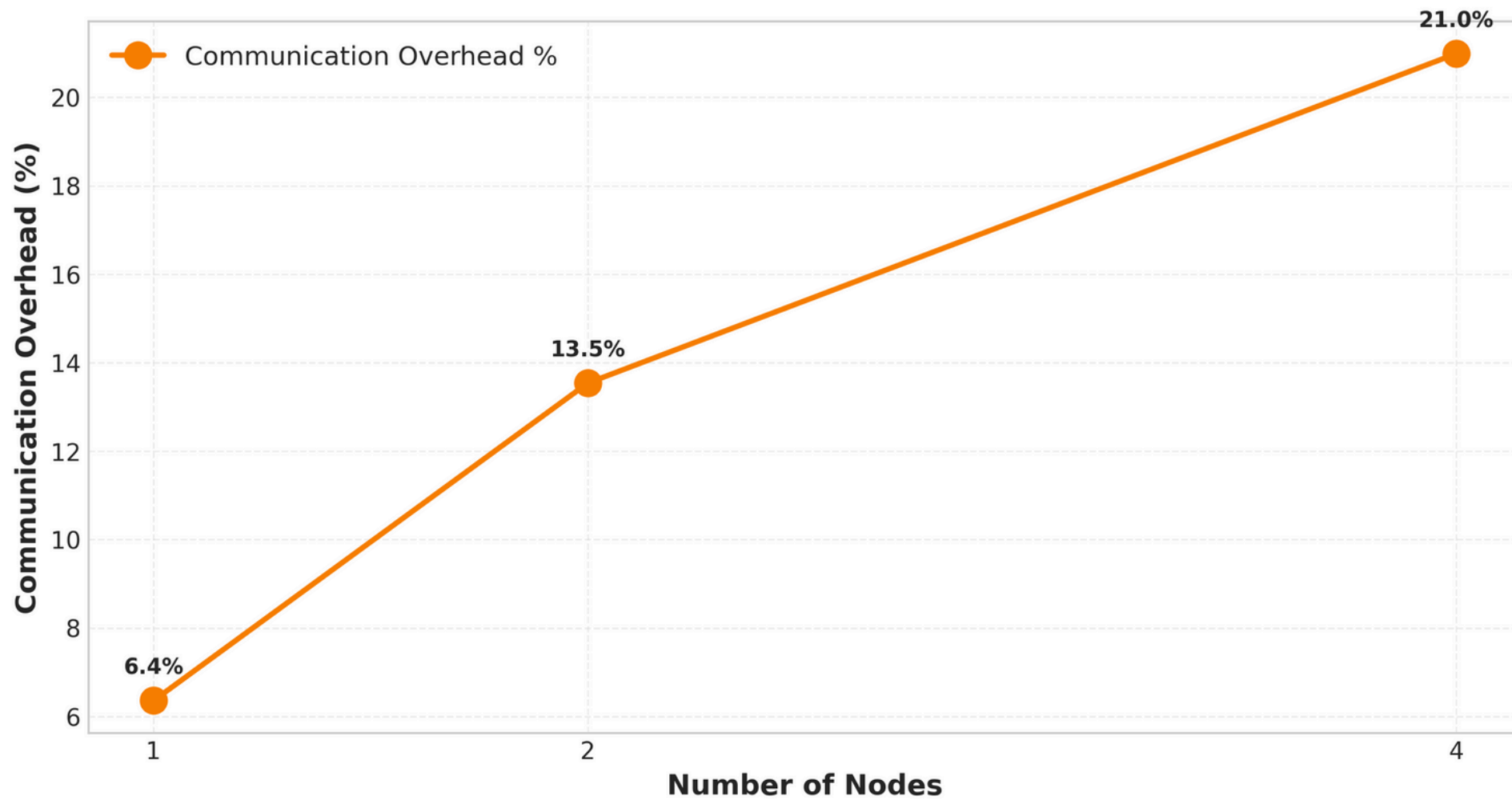
OPEN MP RESULTS

OpenMP Speedup vs Number of Threads
1 Node, 1 MPI Task, Variable Threads

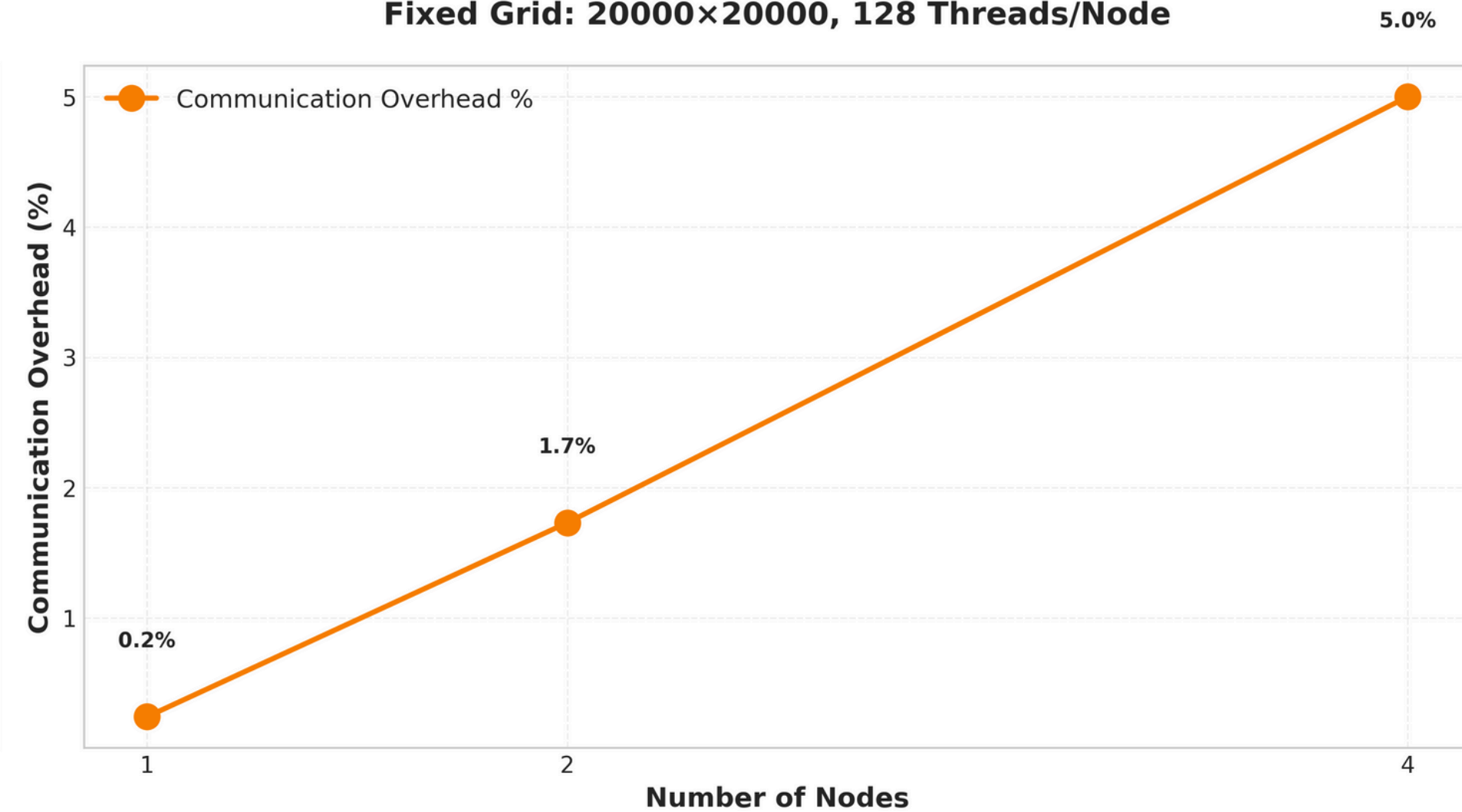


MPI STRONG SCALING

MPI Strong Scaling: Communication Overhead
Fixed Grid: 10000×10000, 128 Threads/Node

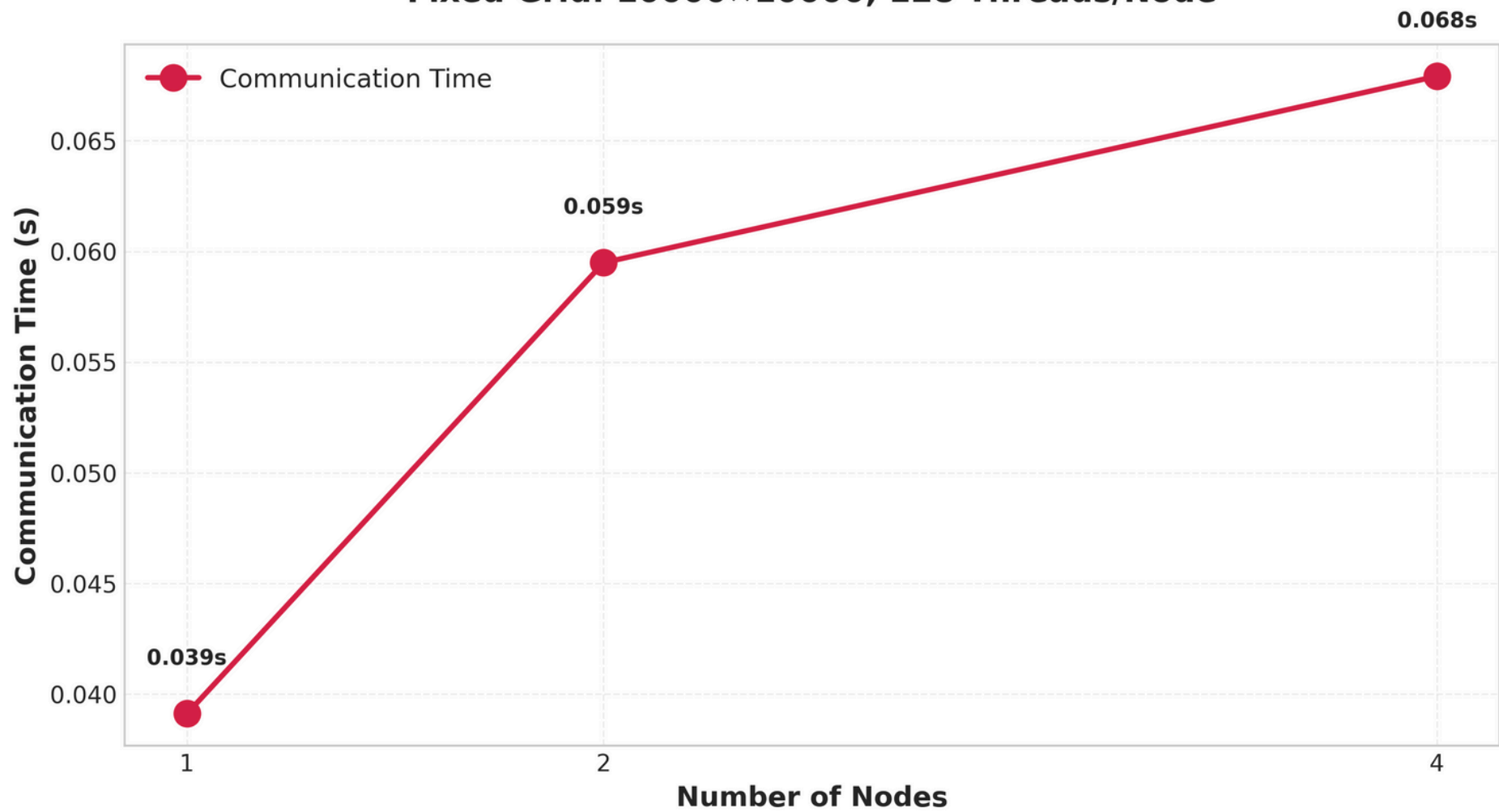


MPI Strong Scaling: Communication Overhead
Fixed Grid: 20000×20000, 128 Threads/Node

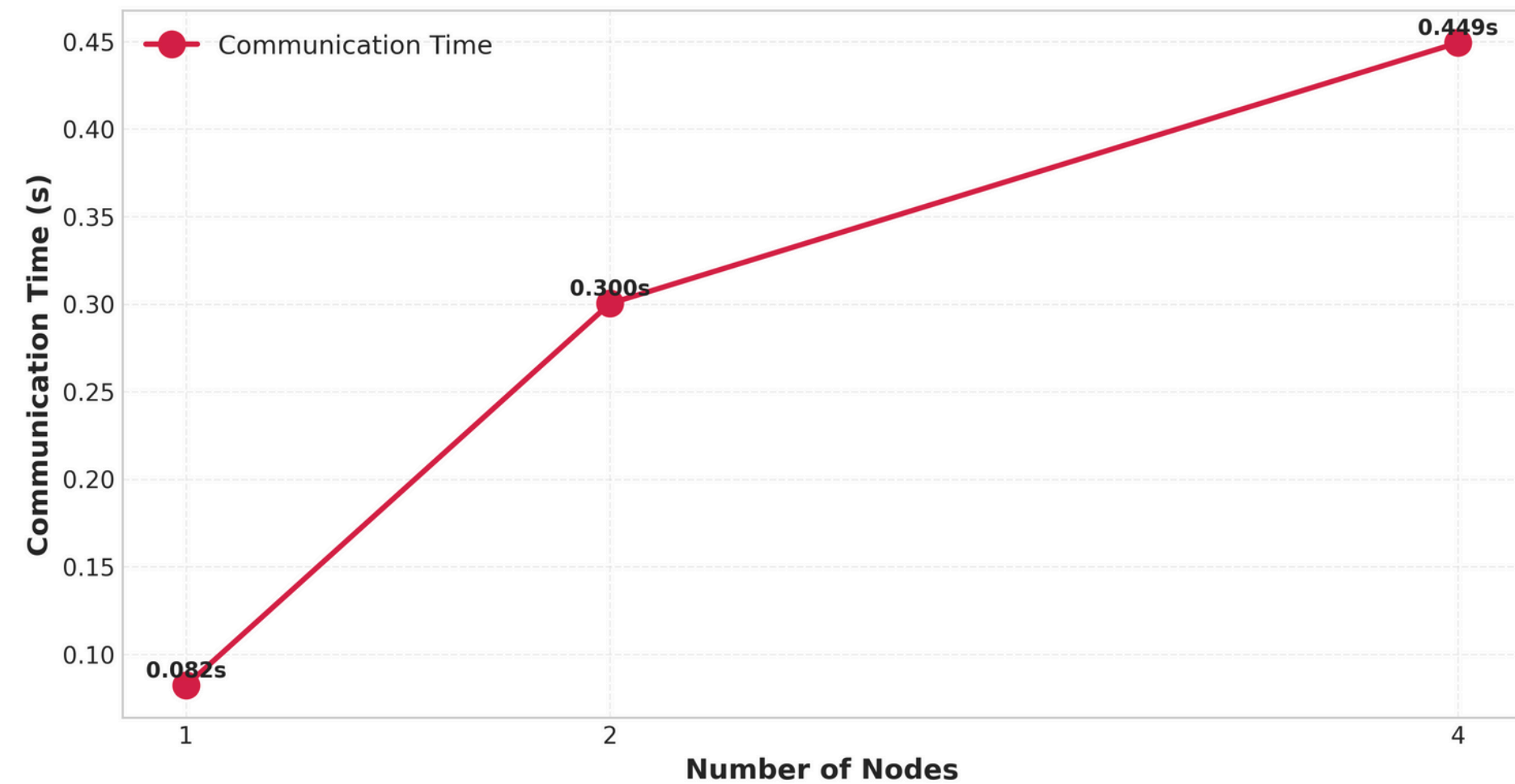


MPI STRONG SCALING

MPI Strong Scaling: Communication Time vs Nodes
Fixed Grid: 10000×10000, 128 Threads/Node

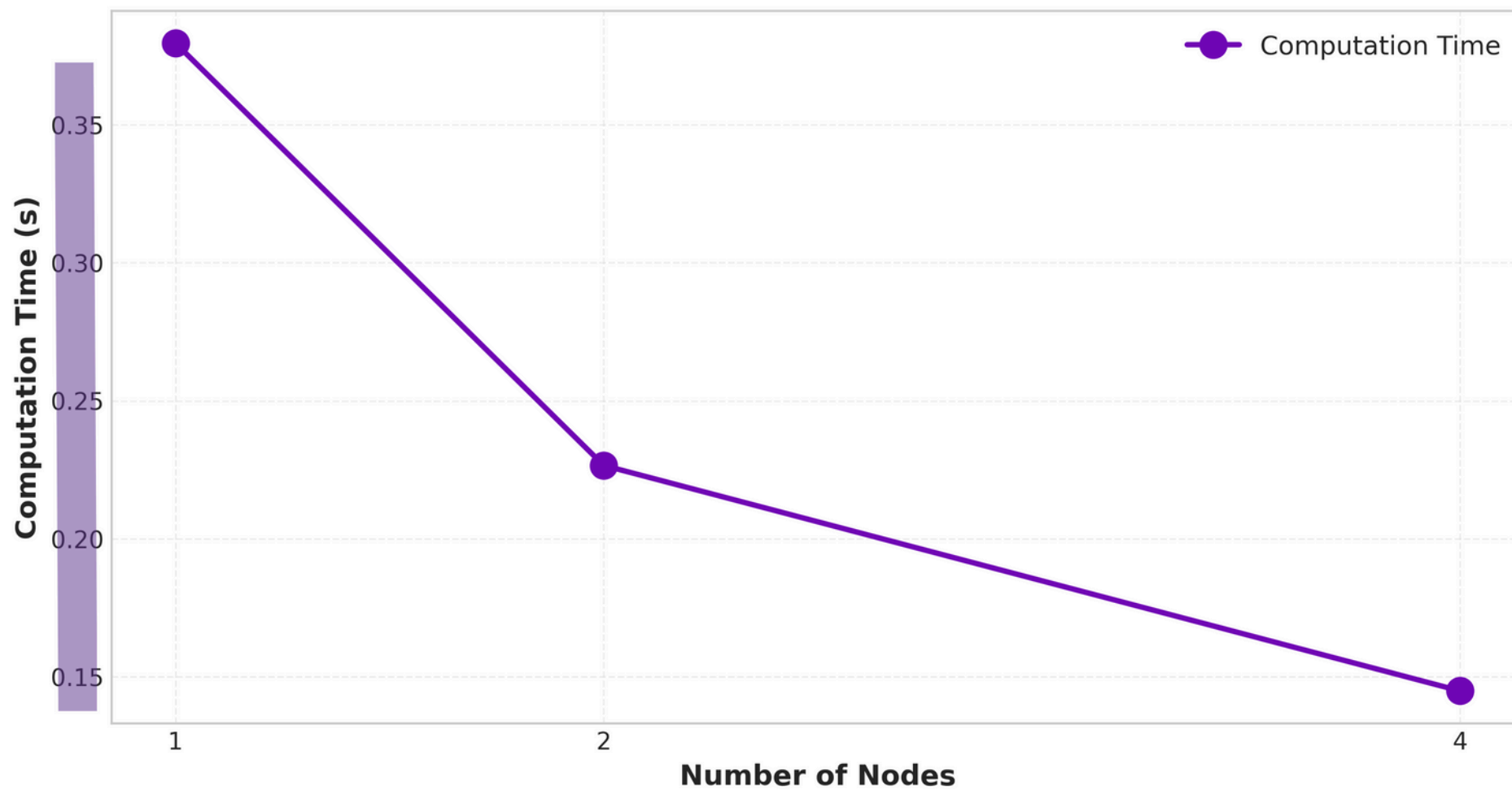


MPI Strong Scaling: Communication Time vs Nodes
Fixed Grid: 20000×20000, 128 Threads/Node

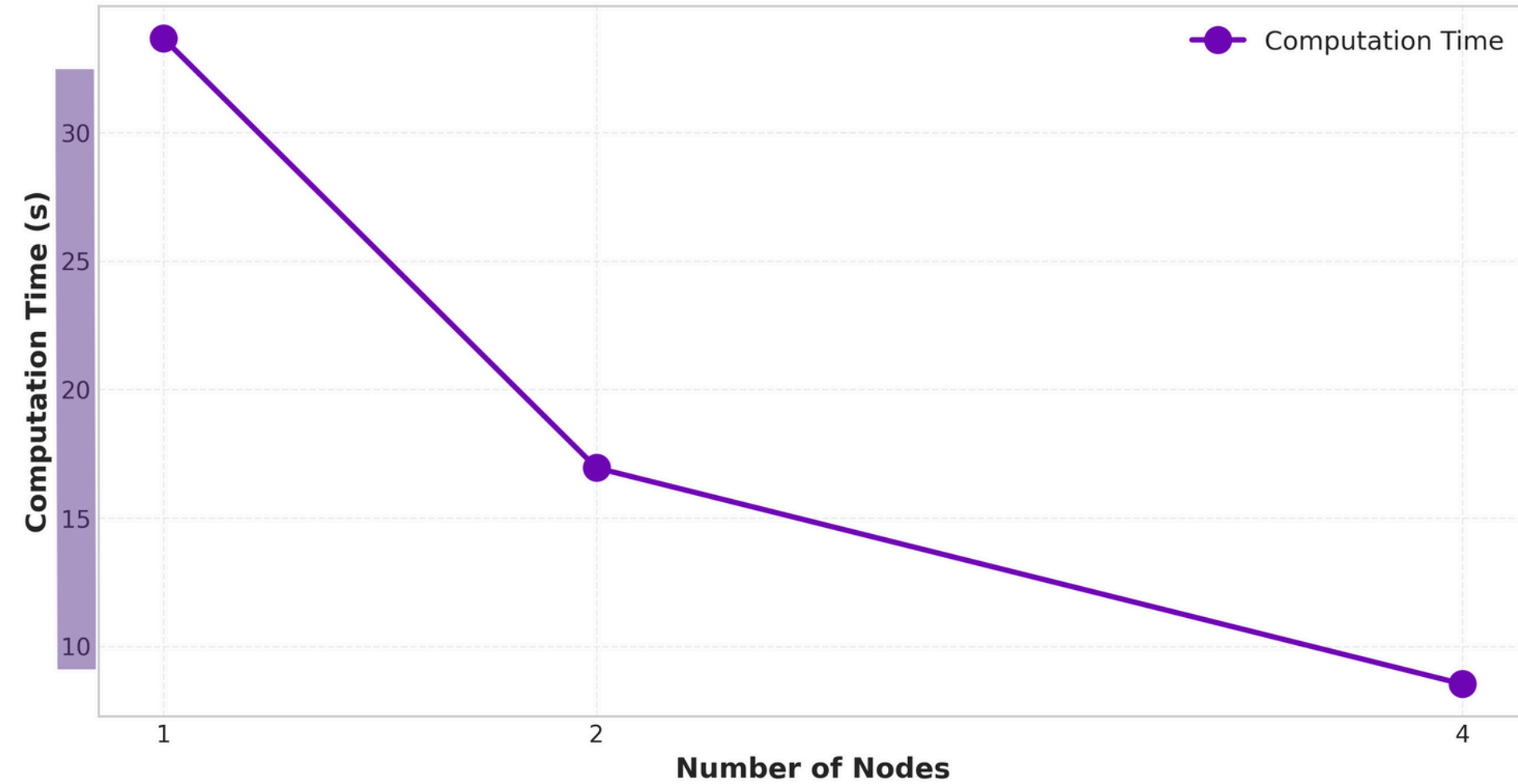


MPI STRONG SCALING

MPI Strong Scaling: Computation Time vs Nodes
Fixed Grid: 10000×10000, 128 Threads/Node

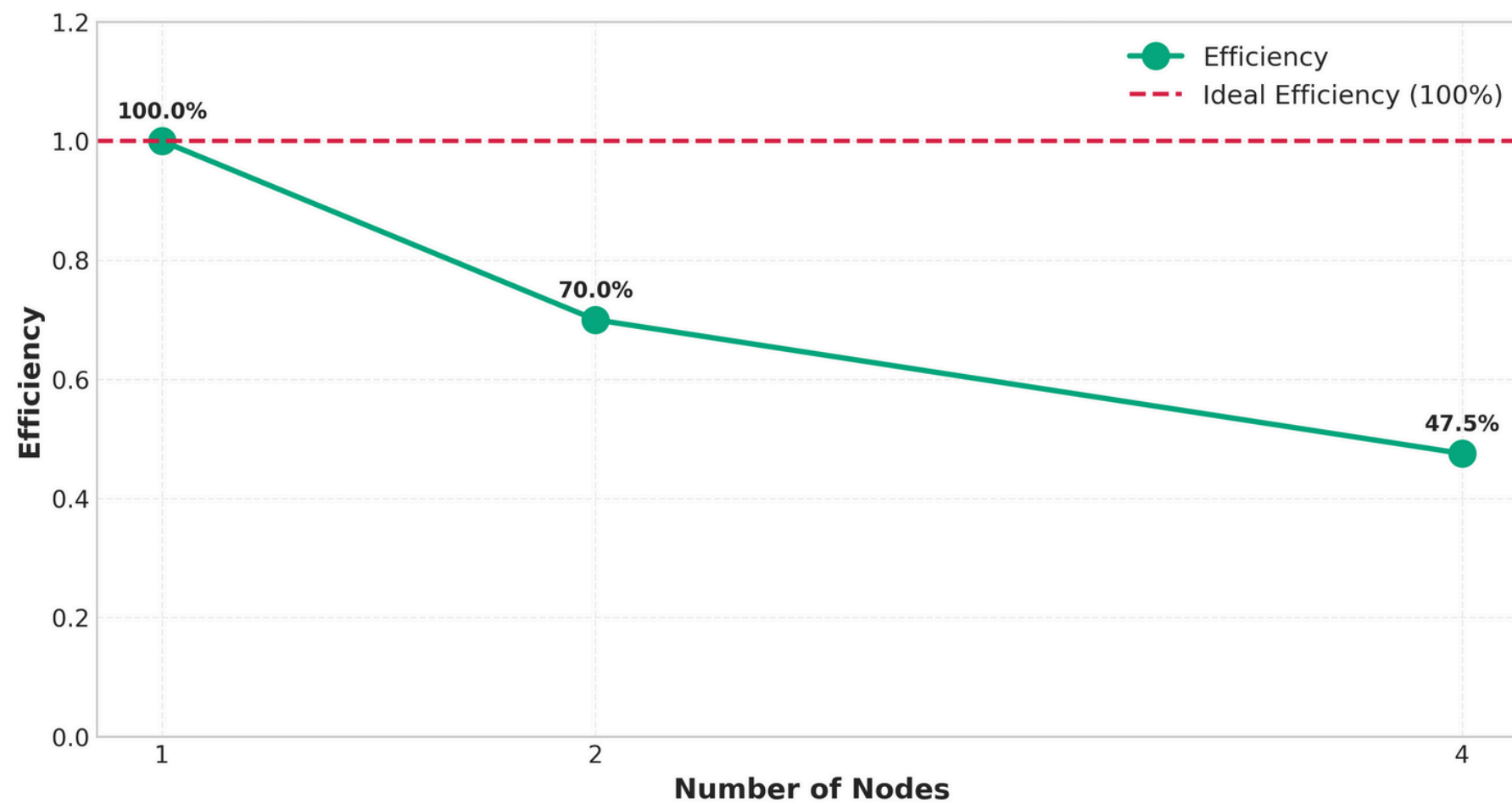


MPI Strong Scaling: Computation Time vs Nodes
Fixed Grid: 20000×20000, 128 Threads/Node

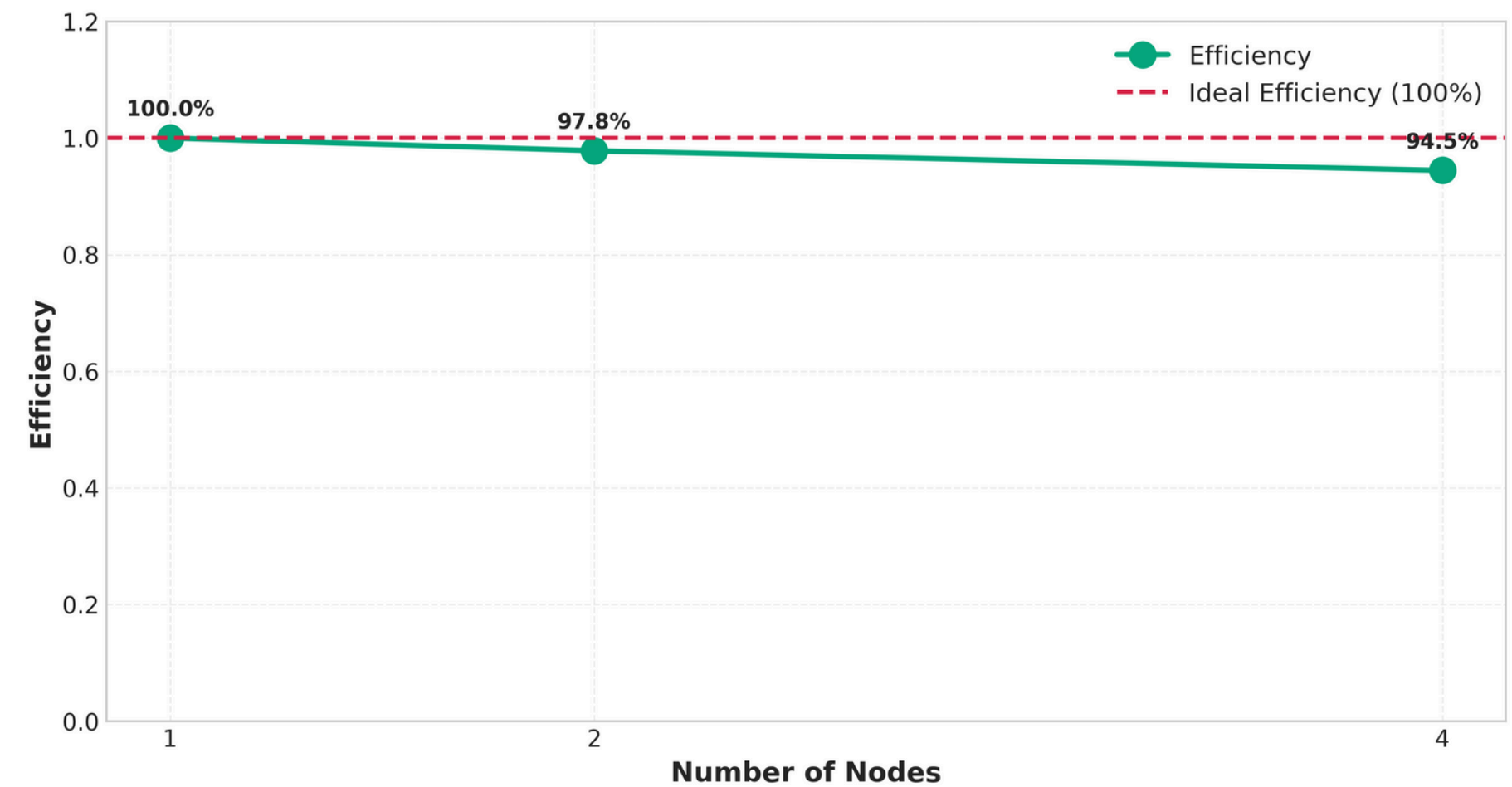


MPI STRONG SCALING

MPI Strong Scaling: Efficiency vs Nodes
Fixed Grid: 10000×10000, 128 Threads/Node

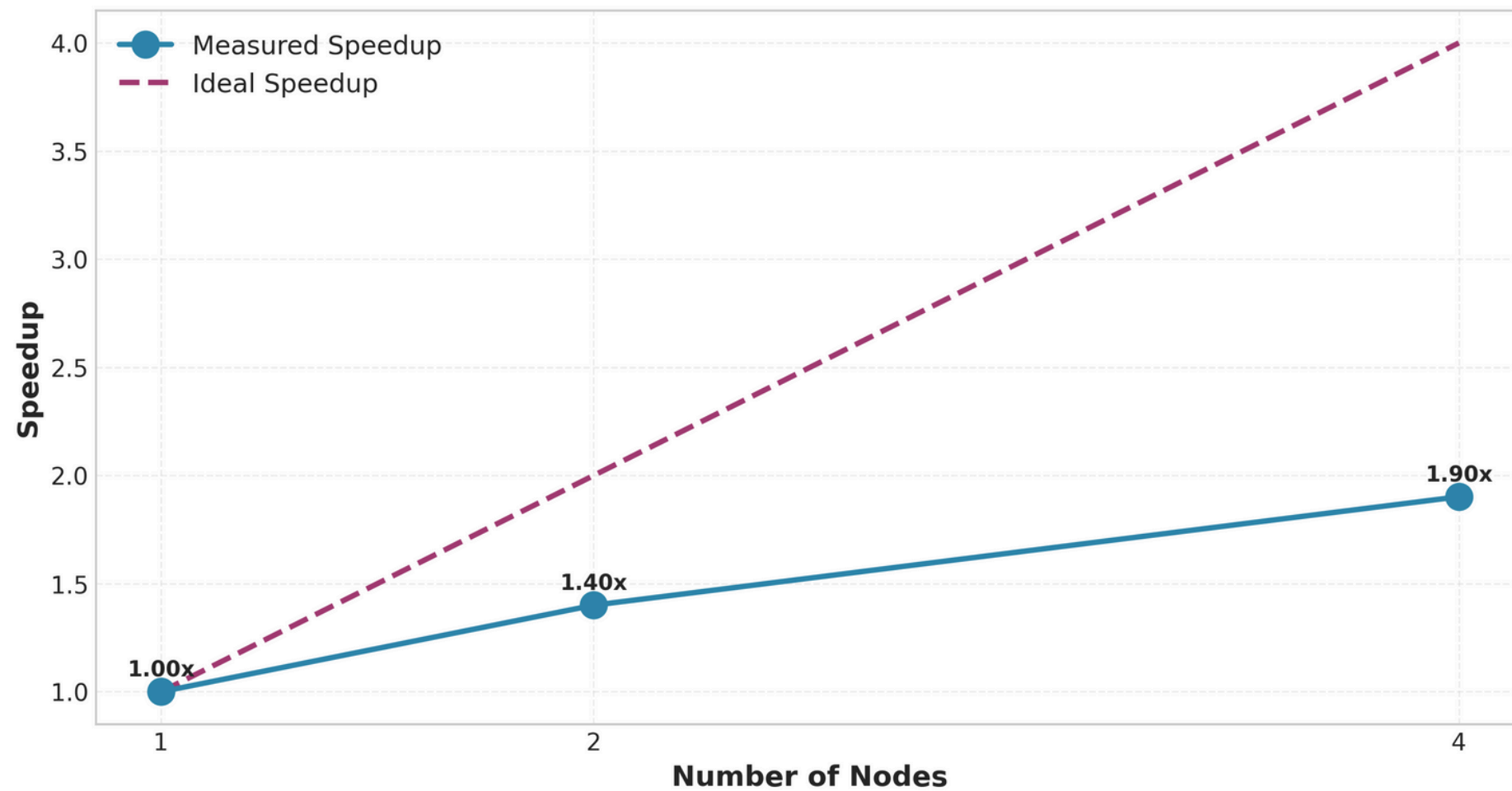


MPI Strong Scaling: Efficiency vs Nodes
Fixed Grid: 20000×20000, 128 Threads/Node

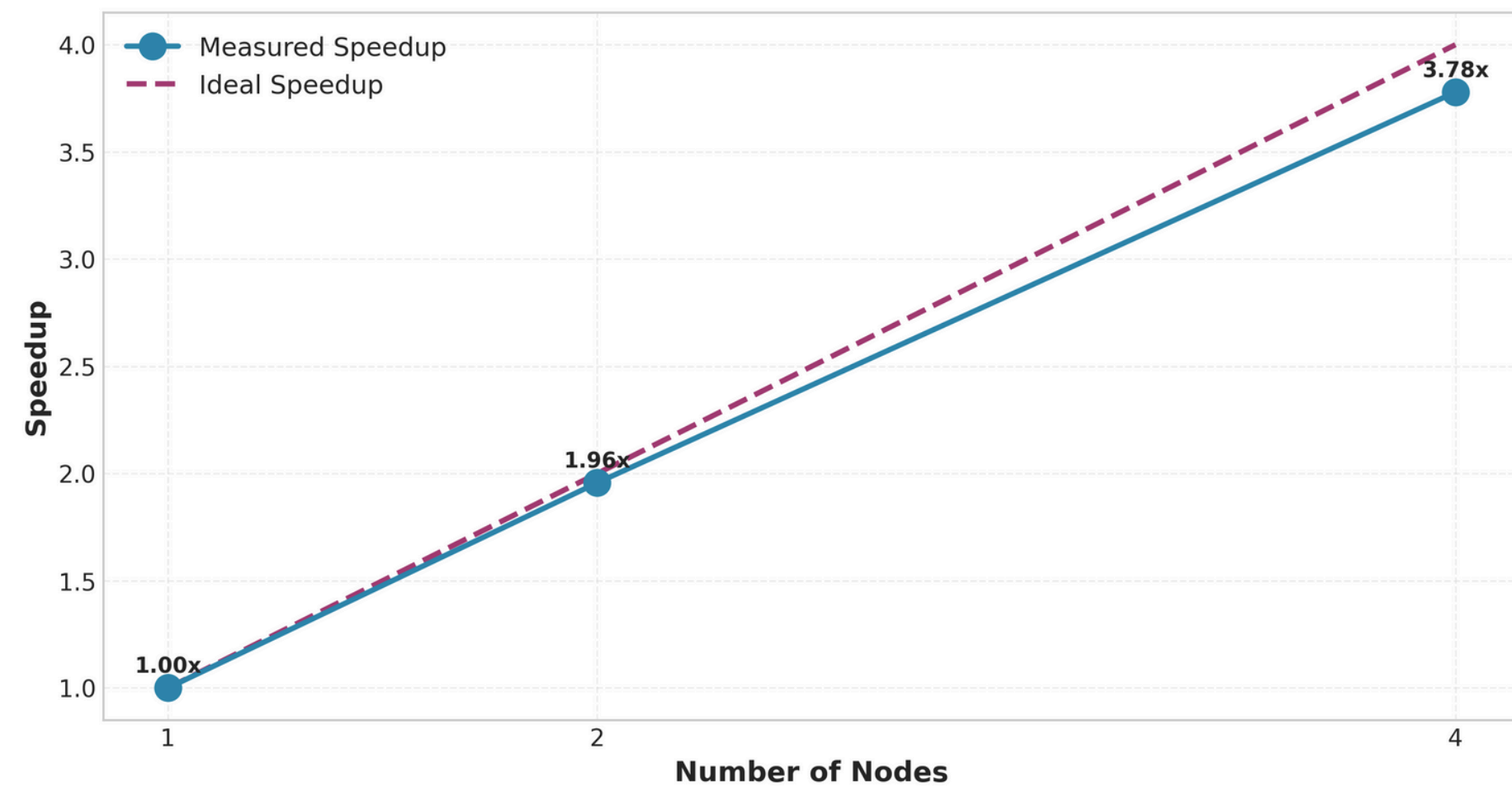


MPI STRONG SCALING

MPI Strong Scaling: Speedup vs Nodes
Fixed Grid: 10000×10000, 128 Threads/Node

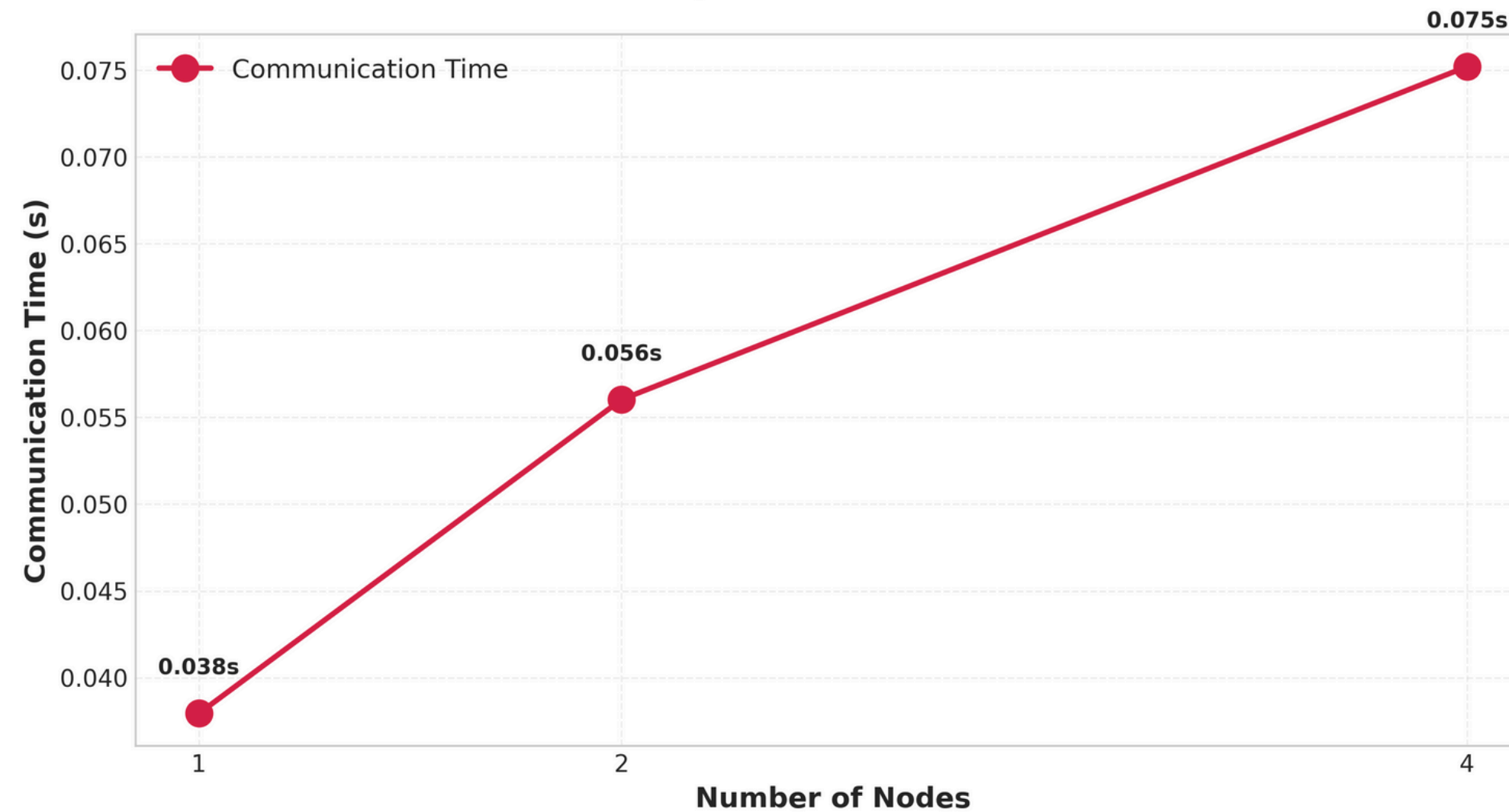


MPI Strong Scaling: Speedup vs Nodes
Fixed Grid: 20000×20000, 128 Threads/Node



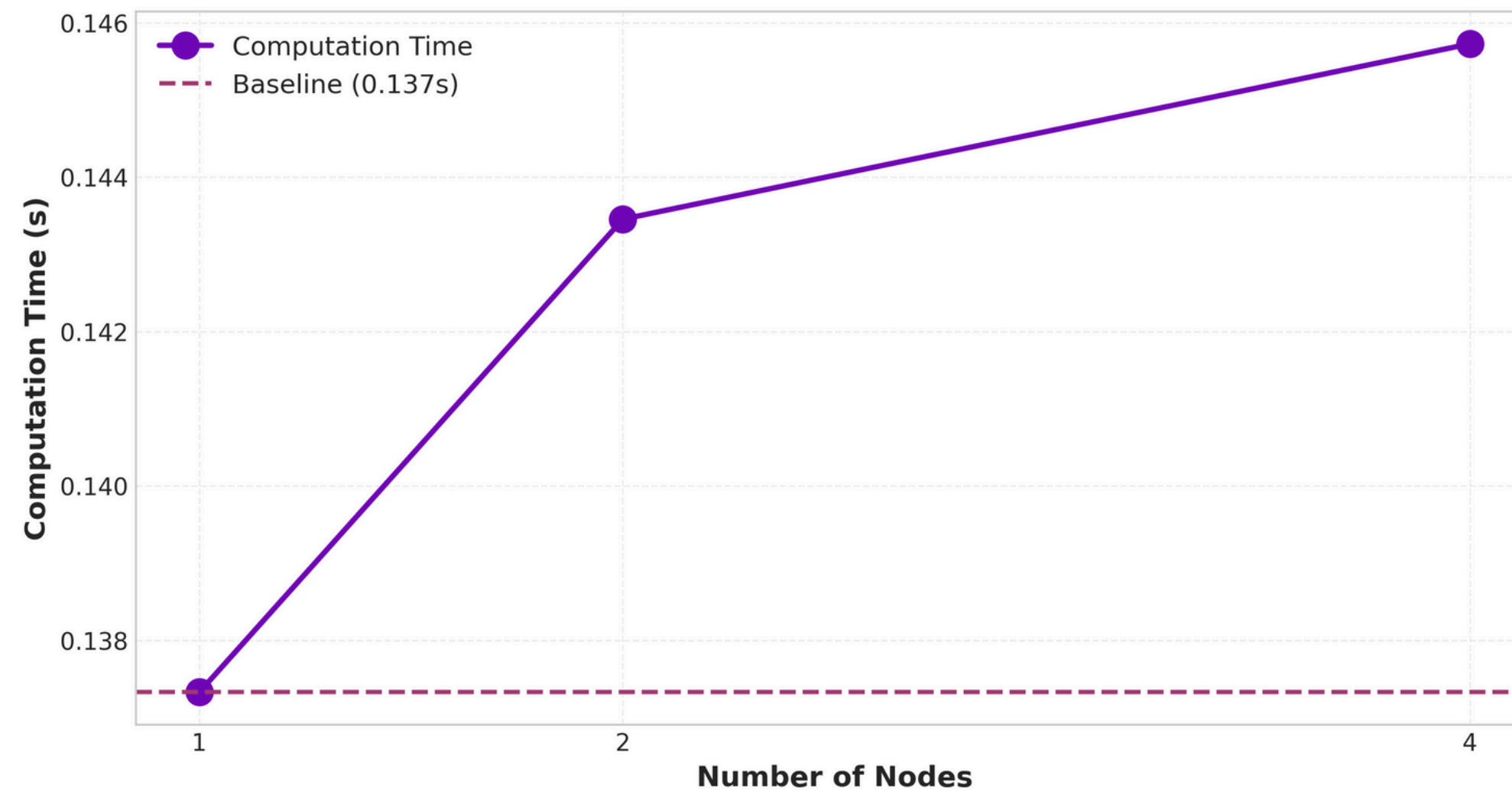
MPI WEAK SCALING

MPI Weak Scaling: Communication Time vs Nodes
~2000×2000 per Node, 128 Threads/Node



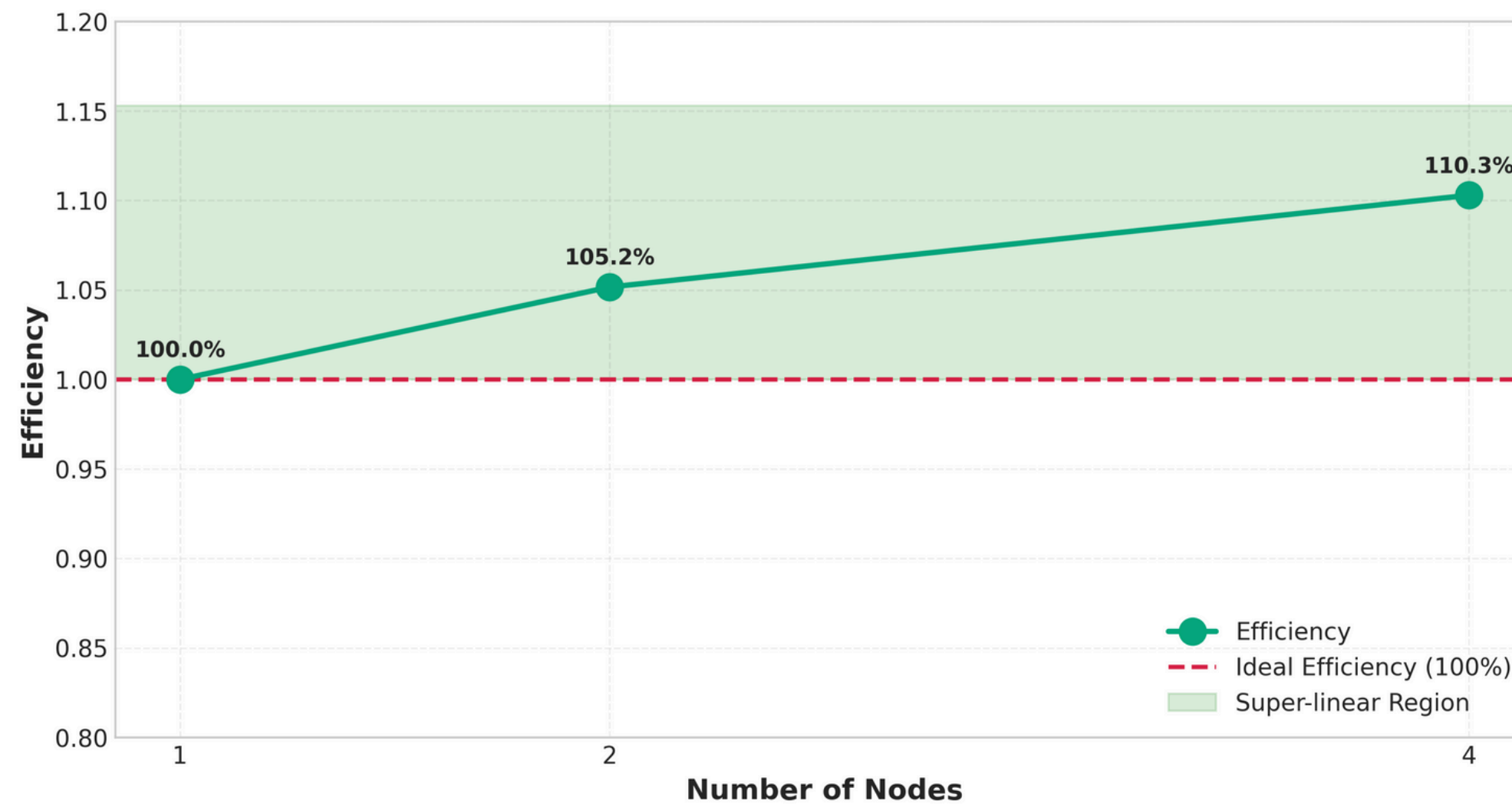
MPI WEAK SCALING

MPI Weak Scaling: Computation Time vs Nodes
~2000×2000 per Node, 128 Threads/Node



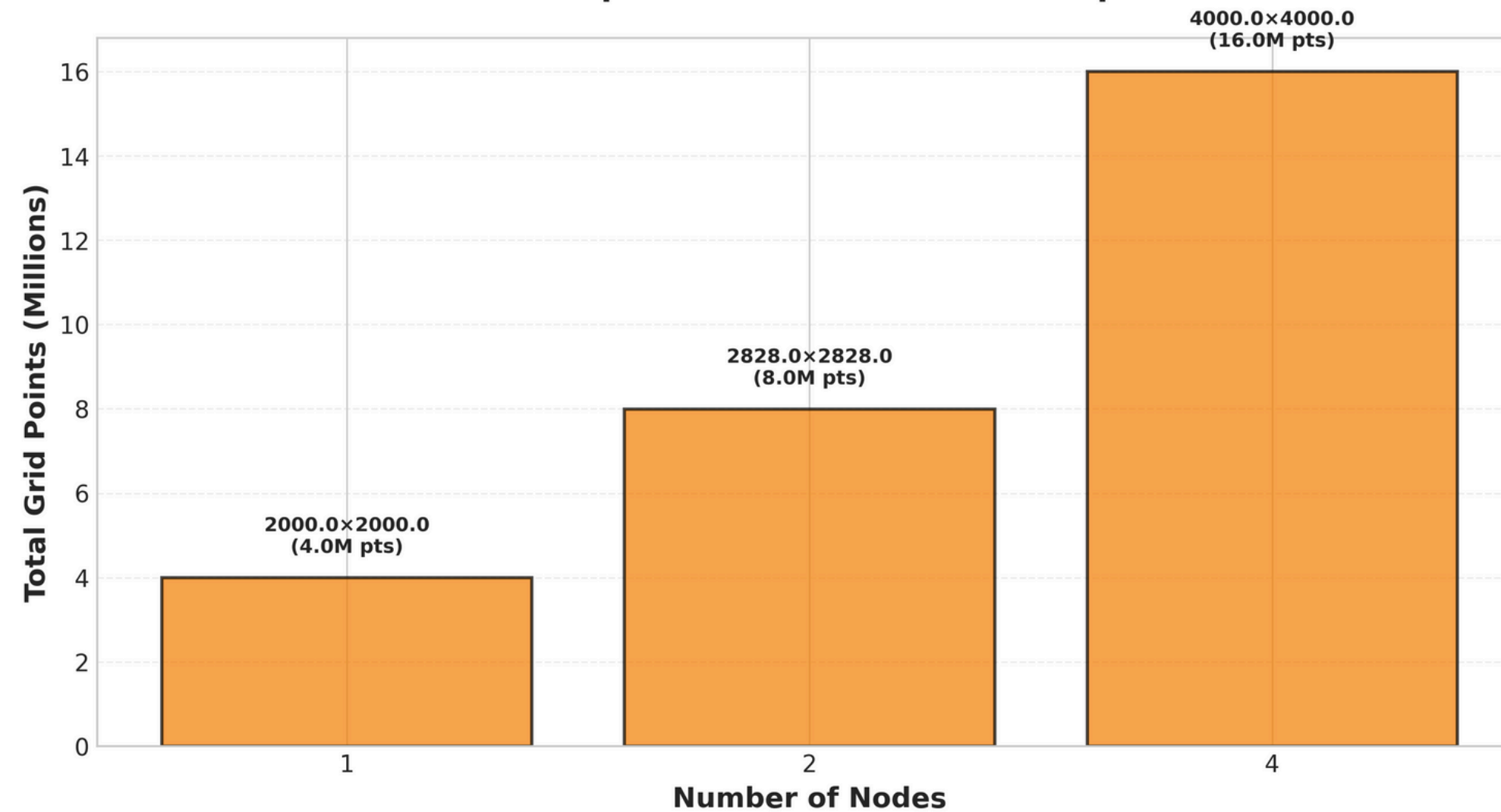
MPI WEAK SCALING

MPI Weak Scaling: Efficiency vs Nodes
~2000×2000 per Node, 128 Threads/Node

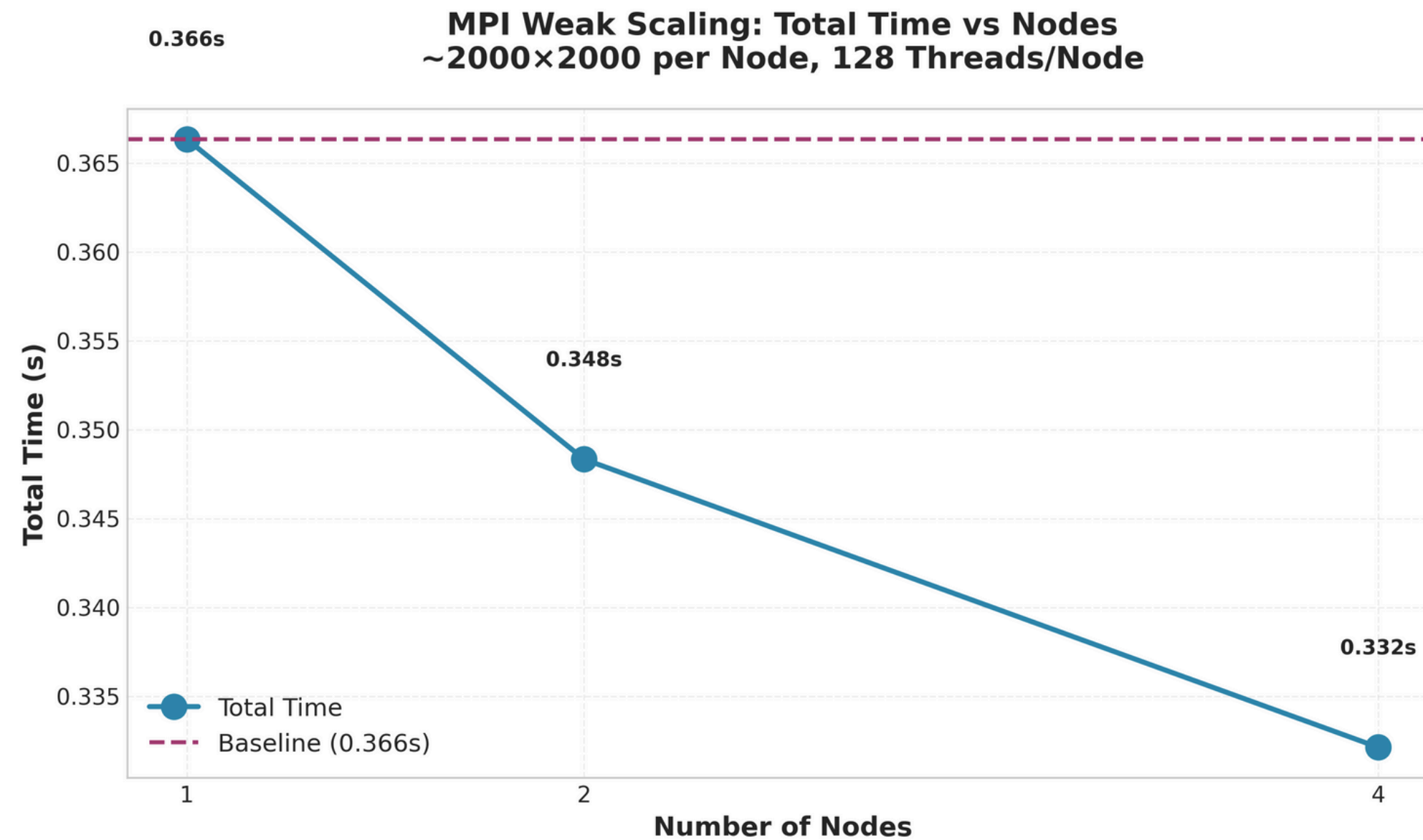


MPI WEAK SCALING

MPI Weak Scaling: Problem Size Growth
~2000×2000 per Node = Constant Work per Node

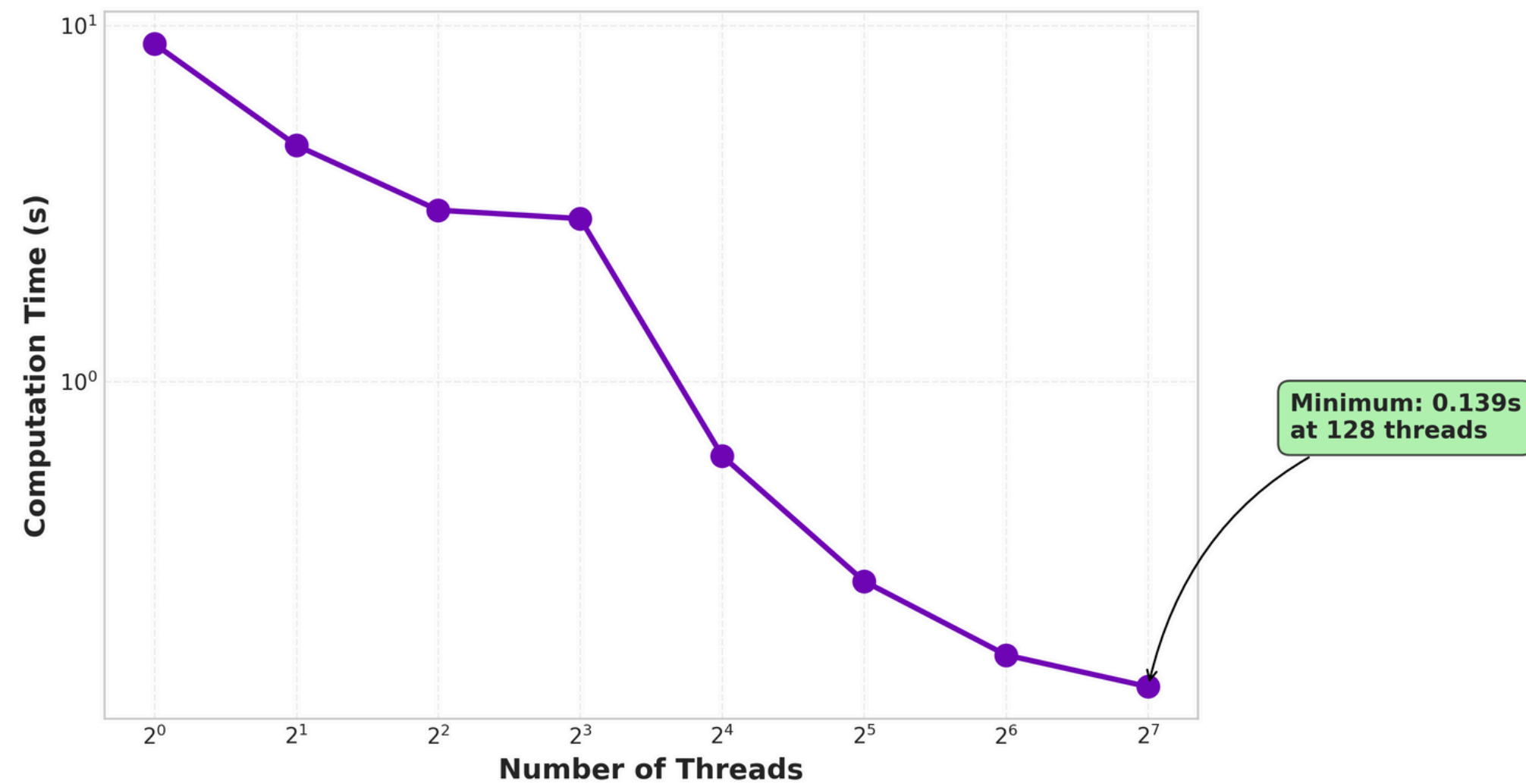


MPI WEAK SCALING



MPI WEAK SCALING

OpenMP Computation Time vs Number of Threads
1 Node, 1 MPI Task, Variable Threads



CONCLUSION

- Instrument: Use `MPI_Wtime( )` before/after communication and computation, collect critical path with `MPI_Reduce(..., MPI_MAX)`
- Strong scaling: Fixed problem, increasing tasks → reveals speedup limits as domains shrink and communication overhead grows
- Weak scaling: Scale problem with task count → shows true utility for larger simulations
- Key metric: Overlap efficiency (interior computation hiding communication latency) should exceed 70%
- Diagnosis: If `comm > 50%`, improve decomposition or overlap; if compute dominates, you're ready to scale

THANK YOU